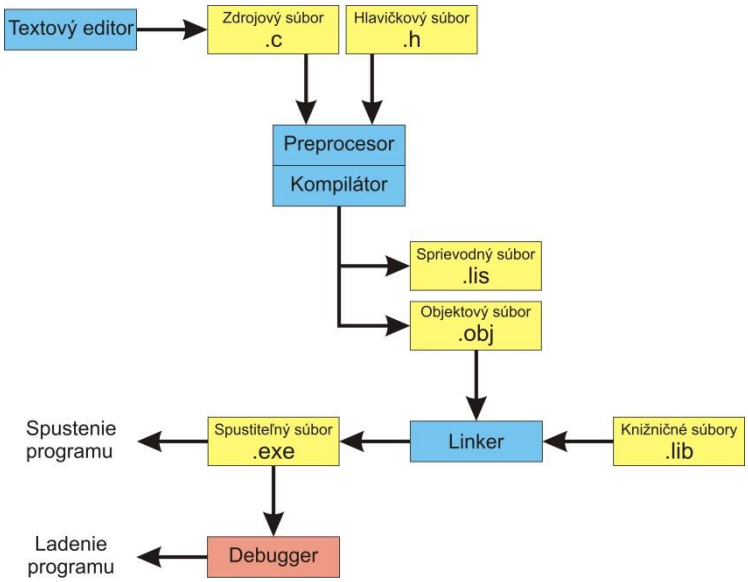
Samostatný text – Základy jazyka C

Lekcia:	***Základy jazyka C***
Modul/tematický celok:	***Robotika***
Téma/predmet:	***Mechatronika***
Odbor:	***Mechanik mechatronik***

Programovací jazyk C vytvorili Brian W. Kernighan a Denis M. Ritchie v sedemdesiatych rokoch 20. storočia. Navrhli ho a implementovali pod operačným systémom UNIX, pričom takmer celý UNIX je napísaný v jazyku C. Tento jazyk je už dlhé roky veľmi obľúbený, a to najmä preto, že je univerzálny, je to jazyk nízkej úrovne a napriek tomu veľmi prehľadný. Univerzálnosť znamená, že programy napísané v jazyku C je možné preložiť pod takmer ľubovoľným operačným systémom a na takmer ľubovoľnej platforme, čiže nie sú obmedzené na konkrétny typ počítača (PC, mikroprocesor atď.) Nízka úroveň tohto jazyka znamená, že programy napísané v jazyku C dokážu veľmi elegantne a rýchlo pracovať s hardvérovými prostriedkami počítača podobne ako strojovo orientované jazyky (prekladané assemblerom). Napriek tomu, že jazyk C je jazykom nízkej úrovne, nie je porovnateľný s assemblerom, pretože príkazy jazyka C nie sú priamymi príkazmi pre procesor. Výhodou jazyka C je jeho oveľa ľahšia čitateľnosť.

V súčasnosti existuje mnoho programovacích jazykov, medzi ktorými je aj skupina takzvaných interpretovaných jazykov (napríklad BASIC, Java alebo Python), ktoré musia mať v príslušnom operačnom systéme nainštalované prostredie na ich spúšťanie – interpreter. Jazyk C do tejto skupiny rozhodne nepatrí, pretože vytvorené programy uložené v súbore s príponou .c sa prekladačom prevedú na strojový kód v spustiteľnom súbore. Takto preloženému kódu už rozumie aj samotný operačný systém bez toho, aby nutne vyžadoval inštaláciu ďalšieho programu v systéme. Proces spracovania programu je naznačený na obrázku číslo 1.



Obr. č. 1 Proces spracovania programu

Samotný program teda môže byť napísaný v ľubovoľnom editore. Pod operačným systémom Windows to môže byť napríklad Dev-C++, Code::Blocks, Borland C++ Builder, MS Visual Studio atď. Tieto editory majú výhodu, že majú v sebe implementovaný aj prekladač. Existujú aj samostatné prekladače a najlepšie to vidieť pri operačnom systéme Linux. Tu sa na prekladanie kódu používa program gcc. Zdrojový súbor s príponou .c, ktorý sa dáva preložiť kompilátoru, môže byť teda vytvorený aj v obyčajnom textovom editore.

Hlavným stavebným prvkom programu napísaného v jazyku C je funkcia. Podrobnejšie informácie o funkciách a ich tvorbe budú uvedené neskôr. Teraz o funkcii stačí vedieť len toľko, že je to pomenovaná postupnosť príkazov a volaní iných funkcií, ktoré vykonávajú požadovanú operáciu alebo činnosť. Okrem toho, že sa dajú vytvárať vlastné funkcie, existuje aj množstvo štandardných funkcií, ktoré sú vopred pripravené v takzvaných štandardných knižniciach prekladača. Tie sú sprevádzané svojimi hlavičkovými súbormi s príponou .h, ktoré obsahujú základné informácie (deklarácie) o funkciách, premenných a iných dátových štruktúrach, ktoré môžu byť v programe použité.

Každý program napísaný v jazyku C musí obsahovať práve jednu funkciu, ktorá má meno main. Po spustení programu sa vždy začínajú vykonávať príkazy od začiatku tejto funkcie. Vo chvíli, keď program pri svojej činnosti dôjde na koniec funkcie main, skončí. Začiatok a koniec funkcie alebo akéhokoľvek bloku programu je ohraničený zloženými zátvorkami { }.

Základná štruktúra jednoduchého programu je naznačená na obrázku číslo 2 a dá sa zhrnúť do nasledovných častí:

1. Pripojenie hlavičkových súborov štandardných funkcií.
2. Definícia makier.
3. Deklarácia (a prípadne definícia) vlastných funkcií.
4. Deklarácia globálnych premenných.
5. Funkcia main.
6. Definícia ostatných funkcií.

```
#include <stdio.h>          // vloženie hlavičkového súboru

// nasledujú definícia makier, napr.:

#define MOJE_MAKRO // za menom makra nasleduje časť kódu v jednom a tom istom riadku

/* nasledujú deklarácie
   a definície funkcií  */

int prva_funkcia();    // deklarácia funkcie

int druha_funkcia()    // deklarácia a definícia funkcie
   {
    // telo druhej funkcie
   }

/* nasledujú deklarácie
   globálnych premenných */

char premenna1;            // deklarácia jednej premennej
int premenna2, premenna3;  // deklarácia dvoch premenných súčasne

// nasleduje funkcia main()

void main()
   {
     printf("Ahoj svet!");
   }

/* tu môžu nasledovať
   definície ostatných
   deklarovaných funkcií:
*/

int prva_funkcia()      // definícia funkcie deklarovanej pred funkciou main()
   {
    // telo prvej funkcie
   }
```

Obr. č. 2 Základná štruktúra programu

Na obrázku číslo 2 je vidieť aj komentáre, ktorými si sprehľadňujeme písaný kód. Jednoriadkový komentár je z ľavej strany uvedený dvomi znakmi lomky a nijako sa neukončuje. Začiatok viacriadkového komentára je uvedený znakmi lomky a hviezdičky a koniec je uzatvorený týmito znakmi v opačnom poradí, čiže hviezdička a lomka. Príkazy, ktoré začínajú znakom mriežky (#), sú príkazmi preprocesora. Ten ich spracuje, vykoná požadované úkony a takto pripravený kód posunie kompilátoru.

Uvedený kúsok kódu je hotovým a funkčným programom, ktorý je možné kompilátorom preložiť a následne spustiť. Program spustený v PC na obrazovku (konkrétne do konzoly) vypíše text: Ahoj, svet! Následne skončí, pretože dôjde na koniec funkcie main. Výpis textu na obrazovku zabezpečuje štandardná funkcia printf, ktorú sme mohli zavolať vďaka tomu, že sme k programu pripojili hlavičkový súbor stdio.h. V tomto súbore sa nachádzajú deklarácie viacerých vstupno-výstupných funkcií, kompilátor teda vie o ich existencii. Potom je pri vytváraní spustiteľného súboru známa presná postupnosť príkazov danej funkcie, ktorá je vložená do výsledného súboru. Koniec programu v operačnom systéme Windows spôsobí aj okamžité zatvorenie konzoly, takže vypísanie textu používateľ nepostrehne. Zatvorenie konzoly môžeme pozdržať tým, že pred koniec programu napíšeme napríklad príkaz na načítanie znaku z klávesnice getchar();. Ten bude čakať na zadanie znaku, pričom konzola bude otvorená a používateľ bude vidieť vypísaný text.

Dôležité je všimnúť si, že za volaním funkcie printf aj getchar sa nachádza bodkočiarka. V jazyku C sa bodkočiarka píše na konci takmer každého príkazu.

Nevyhnutnou súčasťou každého programu je možnosť a schopnosť pracovať s pamäťou. Pamäť si môžeme predstaviť ako súvislý odkladací priestor núl a jednotiek, pričom blok týchto ôsmich binárnych číslic sa nazýva bajt. Ak chceme do pamäte uložiť nejakú hodnotu, napríklad číslo 800, potrebujeme v pamäti vyhradiť priestor veľkosti aspoň 2 bajty a potrebujeme vedieť miesto, kam sa táto hodnota v pamäti uloží, čiže jej adresu v pamäti. Pri písaní programu v jazyku C sa používajú symbolické mená objektov uložených v pamäti, ktoré sa nazývajú identifikátory. Tieto identifikátory majú v sebe ukrytú už aj adresu premennej v pamäti, takže v jednoduchých prípadoch nie je potrebné sa ňou zaoberať. Premennú, do ktorej chceme uložiť naše číslo 800, nazvime napríklad identifikátorom hodnota. Na obrázku číslo 3 je uvedené, ako treba deklarovať premennú hodnota, aby mala vyhradený priestor 2 bajty (aj deklarácia je príkazom, takže je za ňou bodkočiarka). Je to takzvaná globálna premenná, pretože nie je deklarovaná vnútri žiadneho bloku (blokom sa nazýva časť programu ohraničená zloženými zátvorkami). Premennú suma, deklarovanú nižšie, môžeme potom nazvať lokálnou premennou. Každá lokálna premenná existuje od svojej deklarácie v bloku až dovtedy, kým program neopustí daný blok. Potom je z pamäte zrušená. Hodnotu premennej je možné inicializovať aj priamo v deklarácii, ako je to urobené pri premennej suma, alebo aj neskôr v programe pomocou príkazu priradenia, ako je to uvedené pri premennej s názvom hodnota.

```
int hodnota;          // deklarácia globálnej premennej

void main()
   {
     int suma = 1200; // deklarácia lokálnej premennej s inicializáciou
     hodnota = 800;   // priradenie čísla k existujúcej premennej
   }
```

Obr. č. 3 Deklarácia premenných

Na rezerváciu miesta v pamäti bol v príklade zatiaľ použitý iba dátový typ int, ktorý naznačuje, že premenná tohto typu bude v pamäti zaberať práve 2 bajty. Zoznam ďalších často používaných typov spolu s popisom v komentári sa nachádza na obrázku číslo 4.

```
char   /* typ pre celočíselné hodnoty veľkosti 1 Byte, rozsah: 0 - 255
          používa sa aj na ukladanie znakov (ASCII) */
short  // typ pre celočíselné hodnoty veľkosti maximálne 2 Byty, rozsah: 0 - 65535
int    // typ pre celočíselné hodnoty veľkosti práve 2 Byty, rozsah: 0 - 65535
long   // typ pre celočíselné hodnoty veľkosti 4 Byty, rozsah: 0 - 4294967295
float  /* typ pre reálne hodnoty veľkosti 4 Byty, kladné aj záporné,
          najväčšie číslo: 3.402823466e+38
          najmenšie číslo: 1.401298464e-45
          (čísla sú uvedené v exponenciálnom tvare) */
double /* typ pre reálne hodnoty veľkosti 8 Byteov, kladné aj záporné,
          najväčšie číslo: 1.7976931348623157e+308
          najmenšie číslo: 5e-324
          (čísla sú uvedené v exponenciálnom tvare) */
void   // prázdny typ, nerezervuje miesto v pamäti
```

Obr. č. 4 Zoznam často používaných dátových typov

S premennými je možné vykonávať aj aritmetické operácie. Zoznam povolených aritmetických operácií je na obrázku číslo 5. Na obrázku číslo 6 je uvedený príklad použitia aritmetických operácií a priradenie výsledku operácie k premennej. Treba si všimnúť, že aj za príkazom priradenia sa píše bodkočiarka. Zároveň je naznačené, ako sa dá pomocou funkcie printf vypísať hodnota premennej na obrazovku. V jednoduchých zátvorkách sú uvedené parametre tejto funkcie. Ako prvý parameter sa tu nachádza reťazec textu uzavretý v úvodzovkách, ktorý sa má vypísať na obrazovku. Pomocou formátovacích značiek uvedených znakom percenta sú v texte vyznačené miesta, kam sa vypíše obsah premenných. Za týmto reťazcom sa potom nachádza zoznam príslušných premenných oddelených čiarkami v takom poradí, v akom sa majú vyskytnúť v texte na mieste formátovacích značiek.

```
+   // sčítanie, napr:  c = a + b;

-   // rozdiel, napr:   c = a - b;

*   // násobenie, napr: c = a * b;

/   /* celočíselné alebo reálne delenie, napr: c = a / b;
       to, či sa použije celočíselné alebo reálne delenie
       záleží na type použitých premenných
    */

%   // zvyšok po celočíselnom delení, napr: c = a % b;

++  /* inkrement o jednotku:
       príklady:
       - nech i = 4; potom po spustení príkazu i++; bude v premennej i hodnota 5
       - nech i = 4; potom po spustení priradenia c = i++; bude v premennej i hodnota 5, ale
         v premennej c bude hodnota 4, pretože hodnota premennej i sa najprv priradí do c,
         a až potom sa inkrementuje
       - nech i = 4; potom po spustení priradenia c = ++i; bude v premennej i hodnota 5 a aj
         v premennej c bude hodnota 5, pretože premenná i sa najprv inkrementuje, a až potom
         sa jej hodnota priradí do c
    */

--  // dekrement o jednotku, platia rovnaké pravidlá ako pri inkremente, napr: i--; alebo --i;
```

Obr. č. 5 Zoznam povolených aritmetických operácií

```
#include <stdio.h>

void main()
   {
     int suma;
     int hodnota = 800;

     suma = 250 + hodnota; // program spočíta 250 a obsah premennej hodnota a výsledok uloží do premennej suma

     printf("Vypocital som: 250 plus %d sa rovna %d", hodnota, suma);

     /* na obrazovku sa vypíše text:
        Vypocital som: 250 plus 800 sa rovna 1050
     */

   }
```

Obr. č. 6 Príklad použitia aritmetických operácií

Formátovacie značky slúžia na to, aby sa hodnota uložená v premennej zobrazia v takom tvare, ako vyžadujeme. Zoznam základných formátovacích značiek sa nachádza na obrázku číslo 7. Dôležité je uvedomiť si aj fakt, že každá premenná je v pamäti uložená ako binárne číslo. Ak ho však chceme zrozumiteľne zobraziť, musíme použiť požadovanú formátovaciu značku. Dokonca aj samostatné znaky, ktoré sa dajú nájsť v ASCII tabuľke, sú v pamäti v konečnom dôsledku reprezentované číselne, a to poradovým číslom v ASCII tabuľke. Ako vidno na obrázku číslo 8, hodnoty do premenných je možné zadávať v rozličných formátoch a na druhej strane môžeme pomocou formátovacích značiek tieto premenné zobrazovať vo formáte, v akom potrebujeme.

```
%c   // znak
%d   // desiatkové číslo typu int
%ld  // desiatkové číslo typu long
%f   // desiatkové číslo typu float
%x   // číslo v hexadecimálnom tvare s malými písmenami
%X   // číslo v hexadecimálnom tvare s veľkými písmenami
%o   // číslo v osmičkovom tvare
%s   // reťazec
```

Obr. č. 7 Zoznam základných formátovacích značiek

```
#include <stdio.h>

void main()
   {
     char p1 = 76;    // desiatkové číslo
     char p2 = 0x4C;  // hexadecimálne číslo (uvádza sa 0x pred číslom), predstavuje 76 v desiatkovej sústave
     char p3 = 0114;  // osmičkové číslo (uvádza sa 0 pred číslom), predstavuje 76 v desiatkovej sústave
     char p4 = 'L';   // znak (dáva sa do apostrofov), jeho poradové číslo v ascii tabuľke je 76

     // všetky premenné p1 až p4 sú v pamäti naplnené rovnakým binárnym číslom

     printf("Vypis roznych premennych %c %c %c %c ", p1, p2, p3, p4);

     /* na obrazovku sa vypíše text:
        Vypis roznych premennych L L L L
     */

     printf("Vypis rovnakej premennej %x %d %c %o ", p4, p4, p4, p4);

     /* na obrazovku sa vypíše text:
        Vypis rovnakej premennej 4c 76 L 114
     */

   }
```

Obr. č. 8. Použitie formátovacích značiek

Keďže sme si ukázali funkciu, ktorá vytvorí textový výstup na obrazovku, bolo by vhodné ukázať základný spôsob, ako do premennej načítať hodnotu. Slúži na to funkcia scanf, ktorá má podobné parametre ako výstupná funkcia printf. Príklad jej použitia je na obrázku číslo 9. Do prvého parametra – reťazca uzavretého v úvodzovkách, stačí vpísať formátovaciu značku, ktorá hovorí o tom, v akom formáte sa očakáva vstup zadaný z klávesnice. Za čiarkou nasleduje ďalší parameter funkcie scanf, pomocou ktorého sa určí, do ktorej premennej sa má zadaná hodnota načítať po stlačení klávesu Enter. Treba si však dávať pozor na jednu odlišnosť oproti funkcii printf. Pred identifikátor premennej tu treba pripísať znak ampersand (čiže znak anglického „and“ &). Funkcia scanf potrebuje túto kombináciu znaku ampersand a identifikátora preto, lebo to predstavuje konkrétnu adresu premennej v pamäti, na ktorú sa hodnota z klávesnice zapíše. V tejto lekcii je to jediná zmienka o použití adresy premennej, pričom zatiaľ o nej netreba vedieť žiadne ďalšie podrobnosti.

Stačí si len zapamätať, že pri použití funkcie scanf treba pred identifikátor premennej pripísať ampersand. Na záver tejto časti ešte poznamenajme, že funkciou scanf je možné načítať aj viacero premenných naraz.

```c
#include <stdio.h>

void main()
   {
     int vstup;

     scanf("%d", &vstup);

     printf(" Bolo zadane cislo %d ", vstup);

     // na obrazovku sa vypíše číslo, ktoré bolo cez klávesnicu zadané

   }
```

Obr. č. 9 Použitie príkazu vstupu z klávesnice

Takmer v každom programe vieme nájsť časti kódu, ktoré predstavujú určité alternatívne cesty, kadiaľ sa beh programu môže uberať, alebo časti kódu, ktoré sa niekoľkokrát cyklicky opakujú. Prvky, ktoré zabezpečujú takéto správanie, sa nazývajú riadiace štruktúry a riadia tok programu na základe vyhodnotenia podmienok, čiže logických výrazov. Hodnota logického výrazu môže byť buď pravda, alebo nepravda. V jazyku C má nepravda číselnú hodnotu nula a pravda má nenulovú číselnú hodnotu. Výrazy sú tvorené premennými, číslami a volaniami funkcií, ktoré môžu byť vzájomne pospájané aritmetickými operáciami, binárnymi operáciami alebo aj operáciami porovnania. Na obrázku číslo 10 sa nachádza zoznam binárnych operácií i operácií porovnania a v komentároch za nimi sú uvedené ich významy a výsledky.

```c
&&   // logický súčin pravdivostných hodnôt (operácia AND), napr.:   (a && b)
||   // logický súčet pravdivostných hodnôt (operácia OR), napr.:    (a || b)
!    // logická negácia pravdivostných hodnôt (operácia NOT), napr.: (a!)

==   // rovnosť hodnôt, napr.:   (x == y)
!=   // nerovnosť hodnôt, napr.: (x != y)
<    // x je menšie ako y         (x < y)
<=   // x je menšie alebo rovné y (x <= y)
>    // x je väčšie ako y         (x > y)
>=   // x je väčšie alebo rovné y (x >= y)
```

Obr. č. 10 Binárne operácie s pravdivostnými hodnotami a operácie porovnania

Kľúčové slová if a if-else umožňujú spomínané vetvenie v programe. Za kľúčovým slovom if sa nachádza zátvorka, do ktorej sa vpíše výraz. Program na tomto mieste zisťuje pravdivosť výrazu.

Ak je výsledkom pravda, vykonajú sa príkazy uzavreté v zložených zátvorkách, ktoré sa nachádzajú hneď za výrazom. Ak je výsledkom logického výrazu nepravda, tieto príkazy sa nevykonajú a program pokračuje až za nimi. Keby sme chceli vykonať príkazy aj v prípade nepravdivého výrazu, použijeme kľúčové slovo else a program by sa choval takto: Ak je výraz pravdivý, vykonajú sa príkazy uzavreté v zložených zátvorkách hneď za výrazom a nevykonajú sa príkazy v zátvorkách za slovom else.
V opačnom prípade, ak je výraz nepravdivý, nevykonajú sa príkazy za výrazom, ale vykonajú sa tie, ktoré nasledujú v zátvorkách hneď za slovom else. Na obrázku číslo 11 je uvedená syntax oboch príkazov a na obrázku číslo 12 je príklad ich použitia.

```
if (výraz)
   {
    prikaz;
    prikaz;
    prikaz;
    ...
   }
```

```
if (výraz)
   {
    prikaz;
    prikaz;
    prikaz;
    ...
   }
else
   {
    prikaz;
    prikaz;
    prikaz;
    ...
   }
```

Obr. č. 11 Syntax príkazov vetvenia

```
void main()
   {
     int a = 5, b = 6;

     if (a == b)
        {
          /* tento blok sa nevykoná,
             pretože výraz 5 == 6 nie je pravdivý
           */
        }
     else
        {
         // tento blok programu sa vykoná
        }

   }
```

Obr. č. 12 Príklad použitia príkazu vetvenia

Ak je treba v programe vykonať blok kódu viackrát, slúžia na to príkazy cyklov. Prvým z nich je príkaz while. Blok kódu, ktorý sa nachádza za týmto kľúčovým slovom, sa nazýva telo cyklu a bude sa vykonávať dovtedy, kým bude platná podmienka pravdivého výrazu. Prvýkrát sa podmienka kontroluje pred spustením tela cyklu a následne sa kontroluje vždy po každom vykonaní tela cyklu.

Znamená to, že ak už na začiatku bude mať výraz v zátvorkách nepravdivú hodnotu, telo cyklu sa nevykoná ani raz. Ďalší typ cyklu sa označuje kľúčovými slovami do-while. Aj pri ňom sa príkazy v tele cyklu vykonávajú cyklicky dovtedy, kým je podmienka platná (splnená), avšak podmienka sa prvýkrát kontroluje až po prvom vykonaní tela cyklu. Pri tomto type sa telo cyklu vykoná vždy aspoň raz. Na obrázku číslo 13 je uvedená syntax príkazu while aj do-while a na obrázku číslo 14 je príklad ich použitia.

```
while (výraz)          do
   {                      {
    príkaz;                príkaz;
    príkaz;                príkaz;
    príkaz;                príkaz;
    ...                    ...
   }                      }
                         while (výraz);
```

Obr. č. 13 Syntax príkazov cyklu

```
#include <stdio.h>

void main()
   {
    int cislo;

    do
      {
       printf(" Zadaj cislo 5: ");
       scanf("%d", &cislo);
      }
    while (cislo != 5);              // kým nezadáme číslo 5, program si ho bude stále pýtať

   }
```

Obr. č. 14 Príklad použitia príkazov cyklu

Slovom for označujeme cyklus so známym počtom opakovaní. Má podobné vlastnosti ako while. To znamená, že sa kontroluje pravdivosť výrazu už na začiatku a telo cyklu sa vykoná iba vtedy, ak je táto podmienka splnená. Podmienka je stanovovaná tak, aby sa cyklus vykonal toľkokrát, koľko potrebujeme. Tým sa líši od cyklu while, pri ktorom vopred nevieme počet opakovaní cyklu. Cyklus for má navyše v sebe zabudovanú možnosť vykonať vopred stanovenú operáciu po každom jednom vykonaní tela cyklu (napr. zvýšenie alebo zníženie hodnoty premennej atď). Na obrázku číslo 15 je uvedená syntax cyklu for a na obrázku číslo 16 je príklad jeho použitia.

```
for (inicializácia_premennej; výraz; operácia)
   {
    príkaz;
    príkaz;
    príkaz;
    ...
   }
```

Obr. č. 15 Syntax cyklu for

```
#include <stdio.h>

void main()
   {
    int i, cislo;

    cislo = 15;

    for (i = 0; i < 5; i++)   // opakuj, kým platí, že i je menšie ako 5
       {
        cislo--;
        printf("%d ", i);
       }

    printf("cislo je %d", cislo);

    /* Program vypíše na obrazovku:
       0 1 2 3 4 cislo je 10
    */
   }
```

Obr. č. 16 Príklad použitia cyklu for

Niekedy sa stane, že je potrebné, aby sa vykonávanie nejakého bloku programu ukončilo skôr. Najčastejšie sa to môže stať pri cykle, ktorý je potrebné ukončiť napriek tomu, že výraz v podmienke cyklu je stále pravdivý. Kľúčové slovo, ktoré ukončuje vykonávanie bloku kódu v riadiacej štruktúre, je break. Program následne pokračuje vo vykonávaní kódu za touto riadiacou štruktúrou. Ďalším kľúčovým slovom, ktoré zasahuje do procesu zbiehania cyklu, je continue. Tento príkaz zabezpečí, aby sa v aktuálnom kroku cyklu nevykonali príkazy nasledujúce za ním. Z tohto príkazu program vykoná skok na začiatok cyklu. Ak ide o cyklus s podmienkou na začiatku (while alebo for), overí sa platnosť podmienky a pokračuje sa vo vykonávaní tela cyklu, kým je výraz pravdivý. Na obrázku číslo 17 je uvedený príklad použitia oboch príkazov. Za oboma príkazmi vždy nasleduje bodkočiarka. Príkazy break a continue sa odporúča použiť len v nevyhnutných prípadoch, pretože znižujú čitateľnosť kódu.

```c
#include <stdio.h>

void main()
   {
     int cislo = 0;

     while(1)         // cyklus, ktorý sa môže opakovať do nekonečna, lebo 1 je vždy pravdivá hodnota
       {              // začiatok tela cyklu
         cislo++;     // inkrement čísla o 1

         if (cislo <= 3)             // keď je číslo menšie alebo rovné 3, vykonaj nasledovný blok:
              {
               printf("%d ", cislo); // vypíš aktuálne číslo na obrazovku
               continue;             // program skočí na začiatok tela cyklu
              }

         if (cislo == 7)             // ak je číslo rovné 7 vykonaj nasledovný blok:
              {
               printf("Koncim, cislo je %d", cislo);     // vypíš číslo
               break;                // cyklus sa na tomto mieste ukončí, program skočí za telo cyklu
              }

         printf("x ");   // cyklus sa sem dostane jedine, keď je číslo väčšie ako 3

       } // koniec tela cyklu

   } // koniec programu

/*
   program na obrazovku vypíše:
   1 2 3 x x x Koncim, cislo je 7

*/
```

Obr. č. 17 Príklad použitia príkazov break a continue

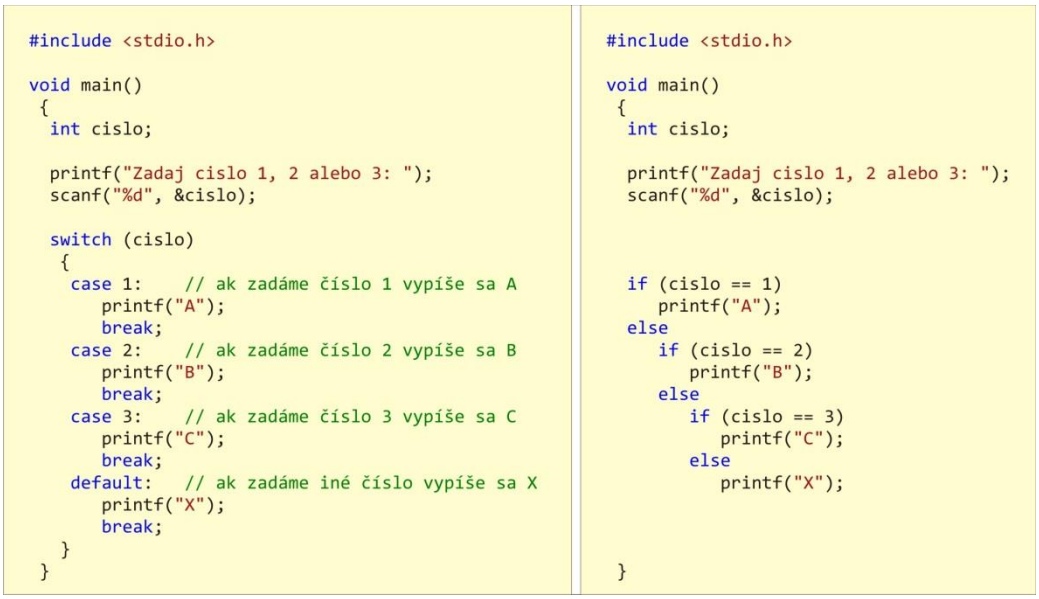
Na ukončovanie bloku kódu existuje ešte príkaz return. Ten sa používa na ukončenie funkcií. V závislosti od typu návratovej hodnoty funkcie sa mení aj syntax tohto príkazu. Informácie o funkciách a ich návratových hodnotách budú uvedené neskôr, teraz stačí vedieť to, že ak funkcia nevracia hodnotu, príkaz return je ukončený bodkočiarkou alebo sa nemusí vôbec použiť. Ak funkcia vracia hodnotu, za príkazom return nasleduje hodnota a za ňou bodkočiarka.

Posledná riadiaca štruktúra, ktorou sa budeme zaoberať, opäť zabezpečuje vetvenie programu. Ide však o viacnásobné vetvenie programu na základe hodnoty sledovaného výrazu za kľúčovým slovom switch. Táto riadiaca štruktúra je tiež ohraničená zloženými zátvorkami. Vnútri sa nachádzajú návestia označené kľúčovým sovom case. Za každým takýmto kľúčovým slovom sa napíše konkrétna hodnota s dvojbodkou a za ňou nasledujú príkazy ukončené príkazom break. Vykonávanie tejto riadiacej štruktúry prebieha takto: Program sa pokúsi v bloku switch nájsť návestie, ktoré má rovnakú hodnotu ako sledovaný výraz. Po nájdení príslušného návestia sa vykonajú príkazy za týmto návestím a na príkaze break program vyskočí z bloku switch. Ak sa nenájde návestie s rovnakou hodnotou, akú má sledovaný výraz, nevykoná sa žiaden príkaz vnútri tejto riadiacej štruktúry a program pokračuje za ňou.

Ak by sme chceli vykonať príkazy v rámci switch aj v prípade, že neexistuje návestie s takou hodnotou, akú má máš sledovaný výraz, použijeme špeciálne návestie default. Príkazy za týmto návestím sa vykonajú vtedy, ak hodnota sledovaného výrazu nezodpovedá žiadnej hodnote pri návestiach označených kľúčovými slovami case. Ak by sme v návestí vynechali príkaz break, program by po vykonaní príkazov tohto návestia nevyskočil z bloku switch, ale pokračoval by vo vykonávaní príkazov nasledujúceho návestia. Switch (so správne použitými príkazmi break v návestiach) si môžeme predstaviť ako viackrát vnorené if-else, no v podmienkach if sa nemôžu testovať nerovnosti (väčší, menší), ale iba konkrétne hodnoty. Na obrázku číslo 18 je uvedená syntax príkazu switch a na obrázku číslo 19 je v ľavej časti príklad jeho použitia. Napravo je program, ktorý vykoná presne to isté, čo program naľavo, len je zapísaný pomocou viackrát vnorenej riadiacej štruktúry if-else. Na tomto príklade je tiež vidieť, že ak za if alebo else nasleduje iba jeden príkaz alebo jedna riadiaca štruktúra, nemusí sa uzatvárať do zložených zátvoriek. Vnorenie viacerých podmienok, ktoré testujú konkrétne hodnoty, program zneprehľadňuje. Preto sa v takomto prípade odporúča použiť switch.

```
switch (výraz_alebo_premenná)
   {
    case hodnota1:
              príkaz;
              príkaz;
         ...
              break;
    case hodnota2:
              príkaz;
              príkaz;
         ...
              break;
    ...
    case hodnotaN:
              príkaz;
              príkaz;
         ...
              break;
    default: príkaz;
              príkaz;
         ...
              break;
   }
```

Obr. č. 18 Syntax príkazu switch



```c
#include <stdio.h>

void main()
 {
  int cislo;

  printf("Zadaj cislo 1, 2 alebo 3: ");
  scanf("%d", &cislo);

  switch (cislo)
   {
    case 1:    // ak zadáme číslo 1 vypíše sa A
       printf("A");
       break;
    case 2:    // ak zadáme číslo 2 vypíše sa B
       printf("B");
       break;
    case 3:    // ak zadáme číslo 3 vypíše sa C
       printf("C");
       break;
    default:   // ak zadáme iné číslo vypíše sa X
       printf("X");
       break;
   }
 }
```

```c
#include <stdio.h>

void main()
 {
  int cislo;

  printf("Zadaj cislo 1, 2 alebo 3: ");
  scanf("%d", &cislo);



  if (cislo == 1)
     printf("A");
  else
     if (cislo == 2)
        printf("B");
     else
        if (cislo == 3)
           printf("C");
        else
           printf("X");



 }
```

Obr. č. 19. Porovnanie použitia switch a vnoreného if-else

Už na začiatku lekcie bolo spomenuté, že programy sú v prevažnej miere zostavené z funkcií. Funkcia je vlastne pomenovaný blok programu, ktorý má svoj vstup a výstup. Vstupy funkcie sa nazývajú parametre a výstup funkcie je návratová hodnota. Na obrázku číslo 20 je uvedený príklad definície funkcie. Keďže funkcia vracia hodnotu, musí byť uvedené, akého typu bude táto hodnota. Ďalej nasleduje identifikátor funkcie a za ním sú v okrúhlych zátvorkách vstupné parametre. Parametre funkcie musia mať takisto uvedený očakávaný typ premennej. Ak funkcia nepoužíva vstupné parametre, okrúhle zátvorky zostanú prázdne (alebo sa do nich píše aj typ void). Tento celý riadok sa nazýva hlavička funkcie. Za ňou nasleduje telo funkcie, čo je vlastne blok programu uzavretý v zložených zátvorkách. V tomto bloku sa spravidla spracujú vstupné parametre a pripraví sa výsledok. Ten je následne odovzdaný ako návratová hodnota funkcie cez kľúčové slovo return. Tento výsledok musí byť takého typu, ako je definovaný typ funkcie. Ak je funkcia definovaná s typom void, nevracia žiadnu hodnotu a program z nej vychádza cez kľúčové slovo return, ktoré je ukončené bodkočiarkou (bez ďalšej hodnoty). Program môže vychádzať z funkcie typu void aj takým spôsobom, že dôjde na koniec tela funkcie. V tomto prípade netreba uvádzať kľúčové slovo return.

```
int sucet (int a, int b)  // hlavička funkcie
    {                     // nasleduje telo funkcie:
      int c;
      c = a + b;
      return c;
    }
```

Obr. č. 20 Príklad definície funkcie

Keď máme takto definovanú funkciu, môžeme ju v programe použiť cez takzvané volanie funkcie. Funkcia sa volá použitím jej identifikátora v programe a vložením požadovaných parametrov (oddelených čiarkami) do okrúhlych zátvoriek za identifikátor. Typy týchto parametrov musia zodpovedať očakávaným typom, ktoré sú definované v hlavičke funkcie. Ak funkcia nemá vstupné parametre, uvedú sa len prázdne okrúhle zátvorky. Návratovú hodnotu funkcie môžeme priradiť k inej premennej alebo ju použiť priamo vo výraze. Ak ju nepriradíme ani nepoužijeme vo výraze, návratová hodnota sa ignoruje. Príklad volania funkcie, ktorá bola definovaná na obrázku číslo 20, je uvedený na obrázku číslo 21.

```
void main()
   {
    int cislo = 5, vysledok;
    vysledok = sucet(cislo, 7); // volanie funkcie, jej návratová hodnota sa priradí k premennej
    // vysledok bude obsahovať hodnotu 12
   }
```

Obr. č. 21 Príklad volania funkcie

Predtým než bude funkcia volaná v programe, musí byť známa aspoň jej hlavička. Celá definícia funkcie (s hlavičkou a telom) môže byť uvedená aj na konci programu, za hlavnou funkciou main, alebo aj v inom súbore či knižnici, ktoré sa pri procese prekladu prilinkujú k programu. Na tento účel teda existujú hlavičkové súbory .h, ktoré sa vkladajú na začiatok programu.

Okrem týchto základných stavebných prvkov jazyka C existuje ešte mnoho ďalších prvkov, ktoré sa často využívajú pri tvorbe programov, sú to napríklad smerníky, polia, dátové štruktúry, makrá, dynamicky alokovaná pamäť, reťazce a práca s nimi alebo aj práca so súbormi atď. Prvky, ktoré boli uvedené a vysvetlené v tejto lekcii, však postačujú na to, aby sme mohli začať písať vlastné jednoduché programy a experimentovať s nimi.